

Conditionals & Iterations

Make infrastructure dynamic — create resources conditionally and loop over collections to stay DRY.

Conditionals

count

for_each

for expressions

Lab 04

Today's roadmap



1 · Conditionals

The ternary operator and conditional resource creation.



2 · count & for_each

Two ways to create many resources — and use each.



3 · for expressions

Transform and filter collections into new lists and maps.



4 · Lab 04

Drive GitHub resources from a map with `for_each`.

01

Conditionals

Choose between values — and decide whether a resource exists at all.

Conditional expressions

Pick between two values based on a condition.

HCL

```
condition ? true_value : false_value
```

- **condition** — evaluates to true / false
- Returns one value or the other
- Nest sparingly — readability first

HCL

```
locals {  
  sku = var.env == "prod" ? "P1v3" : "B1"  
}  
  
resource "azurerm_service_plan" "app" {  
  sku_name = local.sku  
}
```

Conditional creation with count

- Every resource has an implicit `count`
`= 1`
- `count = 0` → the resource is **not created**
- Combine with a bool variable for feature flags

💡 Name the flag clearly: `create_bastion`
beats `flag2`.

HCL

```
variable "create_bastion" {  
  type    = bool  
  default = false  
}  
  
resource "azurerm_bastion_host" "b" {  
  count = var.create_bastion ? 1 : 0  
  # ...  
}
```

02

Iterations

count, for_each and the for expression — Don't Repeat Yourself.

count

Create N identical instances; each gets an index from `count.index`.

- Great for **identical** resources
- Referenced by position: `res[0]`
- Removing a middle item shifts the rest ⚠

HCL

```
resource "azurerm_resource_group" "rg" {  
  count      = 3  
  name       = "app-${count.index}-rg"  
  location   = "westeurope"  
}
```

for_each

Iterate a map or set — each instance is keyed, not positional.

- Best when items are **distinct**
- Stable: removing one doesn't disturb others
- Access `each.key` and `each.value`

HCL

```
variable "rgs" {  
  type = map(string) # name => location  
}  
  
resource "azurerm_resource_group" "rg" {  
  for_each = var.rgs  
  name     = each.key  
  location = each.value  
}
```

The for expression

Build a new list or map by transforming or filtering another.

<> Different from `for_each`: it produces a **value**, not resources.

HCL

```
locals {  
  upper_names = [  
    for n in var.names : upper(n)  
  ]  
  prod_only = {  
    for k, v in var.envs : k => v  
    if v == "prod"  
  }  
}
```

count vs. for_each vs. for

CONSTRUCT	USE WHEN...
count	You need several identical resources, keyed by index
for_each	You iterate distinct items from a list/map/set
for	You transform, filter or build a new list/map value

⚠ Prefer **for_each** over **count** for named things — it avoids the "index shift" trap on rem

Outputs over collections

You can't reference a looped resource as a single object — match how you created it.

- **count** → index into a list: `rg[0].id`
- **for_each** → key into a map:
`rg["app"].id`

HCL

```
output "rg_ids" {  
  value = {  
    for k, r in azurerm_resource_group.rg :  
      k => r.id  
  }  
}
```

 LIVE DEMO

Loop over a map with `for_each`

Create several GitHub resources (labels / repos) from a single map — then add and remove keys to watch the plan react.

Demo · 04 **Provider** · GitHub

🔗 LAB 04 • 30-40 MIN

Dynamic GitHub resources with `for_each`

Drive a set of repository labels (or collaborators) from a map variable — zero cloud cost, instant feedback on your own repo.

You'll practise: `for_each` · `each.key/value` · conditional `count` · outputs over maps

Key takeaways

✓ Ternary chooses values

`count = 0` chooses whether a resource exists.

✓ `for_each` for named things

Stable keys beat positional `count`.

✓ `for` builds values

Transform & filter collections; keep code DRY.

✓ Next up

Session 05 — state backends & Azure a



Questions?

We now leave GitHub behind and head to
[Azure](#) — Session 05.